2nd term

2019- 2018

الفرقة الثانية كهرباء

الفارس

Java

NEW 2019

نعمل لخدماتكم منذ عام 2000

العنوان : كوبري الجامعة - موقف منيا القمح - أسفل قاعة علاء الدين

للإستعلام ت / 01221732228 — 01016772044

السنتر ت :- 01098333260

للشكاوى والمقترحات :- 01224232288

للتواصل عبر الفيس / Al Fares

ABSTRACT CLASSES AND INTERFACES

Center Elfares
كوبري الجامعة - أسفل قاعة عدلي الدين
01221733222 - 01091533988

Abstract Classes

Abstract Class is a class has function definition without implementing these function, these function in their definition add abstract word.

Abstract Class هى فصيله تحتوى على مجموعه من الدوال يتم تعريفها وعدم وضع أكواد بداخلها وهذه الدوال يتم تعريفها إلا ع أنها Abstract Method

GeometricObject
- color : String
- filled: boolean
- GeometricObject( )
- GeometricObject(color : String, filled: boolean)
- getColor():String
- setColor(String color)
- isFilled(boolean filled)
- setFilled():Boolean
- toString():String
- getArea():double
- getPerimeter():double

Center Elfares
كوبري الجامعة - أسفل قاعة عدلي الدين
01221733222 - 01224232288

فعند أنشاء هذه ال Class فعند أرجاع ال Area أو Perimeter هتحسبه لاى شكل من الاشكال الهاتسبه فلا ندرى لان هذه الاشكال سوف تكون مختلفه ومساحتها ومحيطها مختلف ولذلك يتم تعريف داله getArea() ع انها Abstract Method وكذلك داله getPerimeter()

ويتم تعريف Abstract Method بالشكل التالى

public abstract returnDataType MethodName ( );

public abstract نوع القيمه الراجعه من الداله أسم الداله ( );

Example

public abstract double getArea();

public abstract double getPerimeter();

وطالما أن ال Class أحتوت ع Abstract Method فتتحول إلى Abstract Class مباشرة

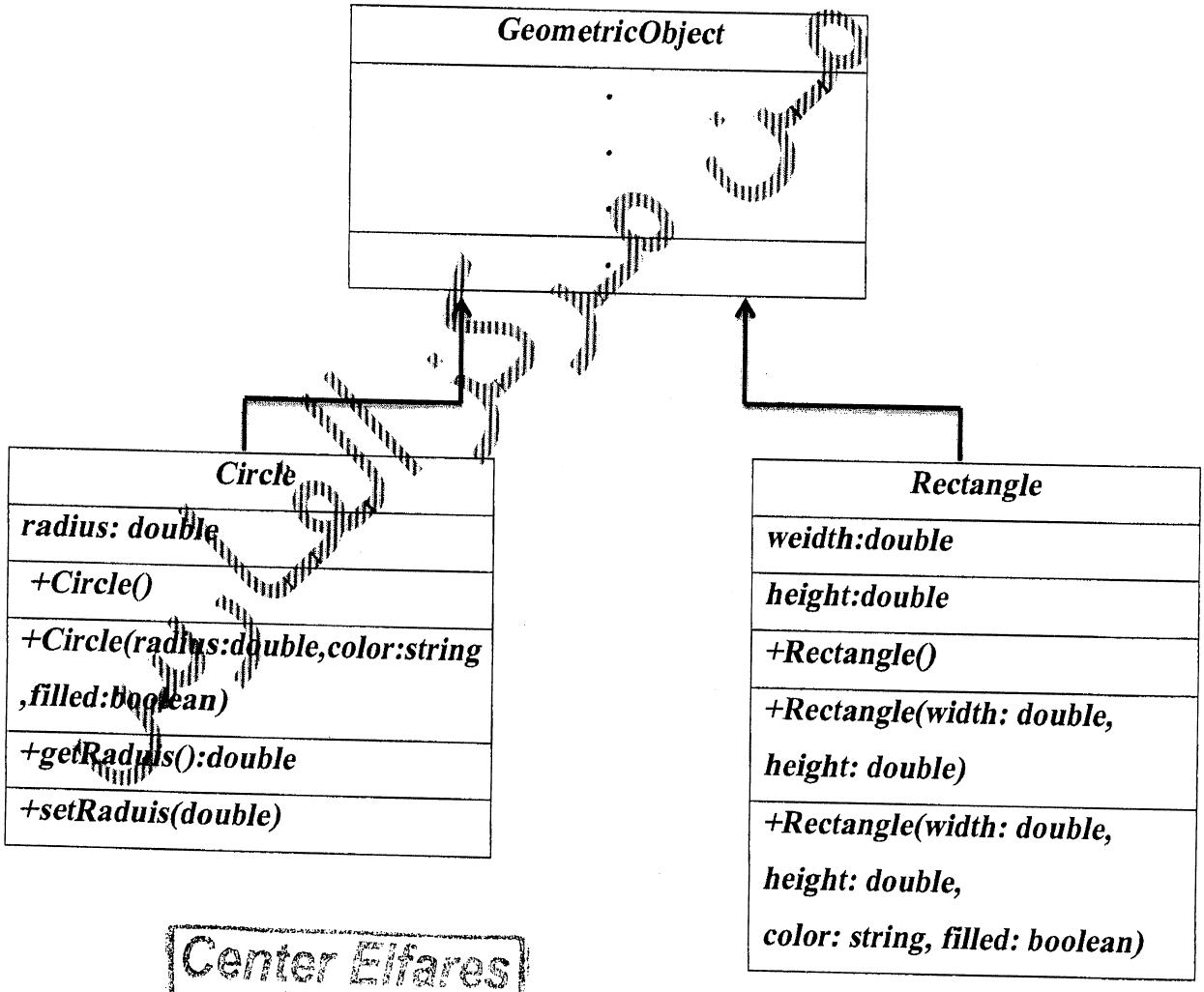
ويتم تعريف ال Class ع انها Abstract بالشكل التالى :-

public abstract class <u>ClassName</u> {

}

Example

public abstract class GeometricObject {}



Center Elfares
كوبرى الجامعة - أسفل قاعة علاء الدين
01221732228 - 01224232288

+getWidth(): double
+setWidth(width: double): void
+getHeight(): double
+setHeight(height: double): void

فعندنا Class تسمى GemetricObject وهى عباره عن Abstract class لانها تحتوى ع Abstract Methods فيكون كتابتها بالشكل التالى

```
public abstract class GeometricObject {
   private String color;
   private boolean filled;
   public GeometricObject() {

   }
   public GeometricObject(String color, boolean filled) {
      this.color = color;
      this.filled = filled;
   }
   public String getColor() {
      return color;
   }
   public void setColor(String color) {
      this.color = color;
   }
   public boolean isFilled() {
      return filled;
   }
   public void setFilled(boolean filled) {
```

Center Efares
كوبري الجامعة - أمام قاعة صلاح الدين
01224232283

```
        this.filled = filled;
    }
    @Override
    public String toString(){
        return "color " + color +"Is Filled? "+filled;
    }
    public abstract double getArea();
    public abstract double getPerimeter();
}
```

وعندنا فصليتن Circle و Rectangle بيرثوا من ال GeometricObject Class
ملحوظه عندما ترث فصليه Sub Class فصيله أخرى Super Class وتكون هذه الفصيله (Super Class)
Abstract فلابد ع الفصيله الوارثة Sub Class أن تعمل Overriding للدوال Abstract Methods
الموجوده داخل ال Super Class
فيكون Circle Class بالشكل التالى

```
public class Circle extends GeometricObject {
    private double raduis;
    public Circle() {
    }
    public Circle(double raduis, String color, boolean filled) {
        super(color, filled);
        this.raduis = raduis;
    }
    public double getRaduis() {
        return raduis;
    }

    public void setRaduis(double raduis) {
        this.raduis = raduis;
```

```
    }
    public  double getArea(){
        return Math.PI*raduis*raduis;
    }
    public  double getPerimeter(){
    return Math.PI*raduis*2;
    }
}
```

وتكون RectangleClass بالشكل التالى

```
public class Rectangle extends GeometricObject {
  private double Width;
  private double height;
   public Rectangle() {
   }
   public Rectangle(double Width, double height) {
      this.Width = Width;
      this.height = height;
   }
   public Rectangle(double Width, double height, String color, boolean filled) {
      super(color, filled);
      this.Width = Width;
      this.height = height;
   }
   public double getWidth() {
      return Width;
   }
   public void setWidth(double Width) {
      this.Width = Width;
```

Center Elfares
كوبري الجامعة - أسفل قاعة عبد الحميد
01221732228 - 01224232228

```
    }
    public double getHeight() {
        return height;
    }
    public void setHeight(double height) {
        this.height = height;
    }
  public double getPerimeter(){
  return 2*(Width+height);
  }
  public  double getArea(){
     return Width *height;
  }
}
```

ال Abstract Class لايمكن أنشاء Object منها

Abstract classes are like regular classes, but you cannot create instances of abstract classes using the new operator. An abstract method is defined without implementation. Its implementation is provided by the subclasses. A class that contains abstract methods must be defined as abstract.

لا يمكن أنشاء Object من ال Abstract Class

ال Abstract Class لابد أن تحتوى علي Abstract Method

ال Abstract Method هى عبارة عن Method بدون بناء

The constructor in the abstract class is defined as protected, because it is used only by subclasses. When you create an instance of a concrete subclass, its superclass's constructor is invoked to initialize data fields defined in the superclass.

ال Constructor بتاع Abstract Class يمكن تعريفه ع أنها Protected لانه لايتم أستدعائه الا داخل ال Sub Class فقط وذلك عند أنشاء Object من ال Sub Class

فائدة ال Abstract Class

لاحظ البرنامج التالى لاختبار الفصائل السابقه

```
public class TestObject {

    public static void main(String[] args) {
        GeometricObject g1 = new Circle(5);
        GeometricObject g=new Rectangle(5,3);
        System.out.println("Are Two Areas Equalirem?? "+ equalArea(g1,g));
        dispalyGeometricObject(g1);
        dispalyGeometricObject(g);
    }

    public static boolean equalArea(GeometricObject x,GeometricObject y){
        return (x.getArea()==y.getArea());
    }

    public static void dispalyGeometricObject(GeometricObject x){
        System.out.println();
        System.out.println("The area is " + x.getArea());
        System.out.println("The perimeter is " + x.getPerimeter());
    }
}
```

Center Elfares
Center Elfares
بيرى الجامعة - أسيل قاعة عزى الدين
01221732228 - 01224232228

هنلاحظ اننا لو عملنا داله بتختبر المساحات بتاعت الاشكال متساويه ولا لا فلو كنا معملناش ال Abstract Class كنا أضصرينا أننا نعمل داله لكل شكلين مختلفين يعنى Circle With Circle و Circle With Rectangle وRectangle ولكن باستخدام ال Abstract class تم تعريف داله واحده وتعريف الاشكال ع انها أى كان Circle او Rectangle وارسالها لهذه الداله وعن طريق Overriding تم

أستدعاء الدوال الموجوده فى ال Sub Class فيتم حساب المساحه لل Circle من Circle Class وكذلك لل Rectangle من Rectangle Class ثم يتم المقارنه بين الرقمين وكذلك داله العرض

An abstract method cannot be contained in a non abstract class. If a subclass of an abstract superclass does not implement all the abstract methods, the subclass must be defined as abstract. In other words, in a non abstract subclass extended from an abstract class, all the abstract methods must be implemented. Also note that abstract methods are non static.

ال Abstract Method لايمكن أن تتواجد داخل Class وهى ليست Abstract و كذلك ال Sub Class اذا لم تقم بعمل Implementation للدوال Abstract كلها الموجوده فى Super Class لابد من تعريفها ع انها Abstract Class

Center Elfares

Interfaces

An interface is a class-like construct that contains only constants and abstract methods.

ال Interface ماهو الا Abstract Class ولكن تحتوى فقط ع Abstract Methods وثوابت

أل Methods الموجوده داخل ال Interface كلها Public Abstract

ال Constants الثوابت الموجوده داخل ال Interface كلها public static

ويتم أنشاؤها كالتالى

```
public interface أسم {
  يتم وضع الثوابت والدوال هنا
}
```

Center Elfares

Example

```
public interface MathInterface {

}
```

مثال قم بعمل Interface به بعض الدوال

MathInterface
Sum(int x , int y):double

subtract(int x , int y):double
multiply(int x , int y):double
devision(int x , int y):double

```
public interface MathInterface {
  double sum (int x,int y);
  double subtract (int x,int y);
  double multiply (int x,int y);
  double devision (int x,int y);
}
```

كل الدوال المعرف داخله كلها public وكمان abstract

تقوم ال Classes الاخرى بوارثته وعمل Implementation لهذة الدوال

وتكون وراثه ال Interface عن طريق كلمة implements

Public class أسم الكلاس implements أسم الانترفيس

Example

```
public class MathImplementation implements MathInterface {
   interface يتم تعريف الدوال الموجوده داخل ال
}
```

ويتم تعريف ال Class بالشكل التالى

```
public class MathImplementation implements MathInterface {
  public double sum (int x,int y){
     return x+y;
  }
  public double subtract (int x,int y){
     return x-y;
  }
  public double multiply (int x,int y){
```

```
    return x*y;
  }
  public double devision (int x,int y){
    return x/y;
  }
}
```

Center Elfares

لماذا ال Interface ؟؟

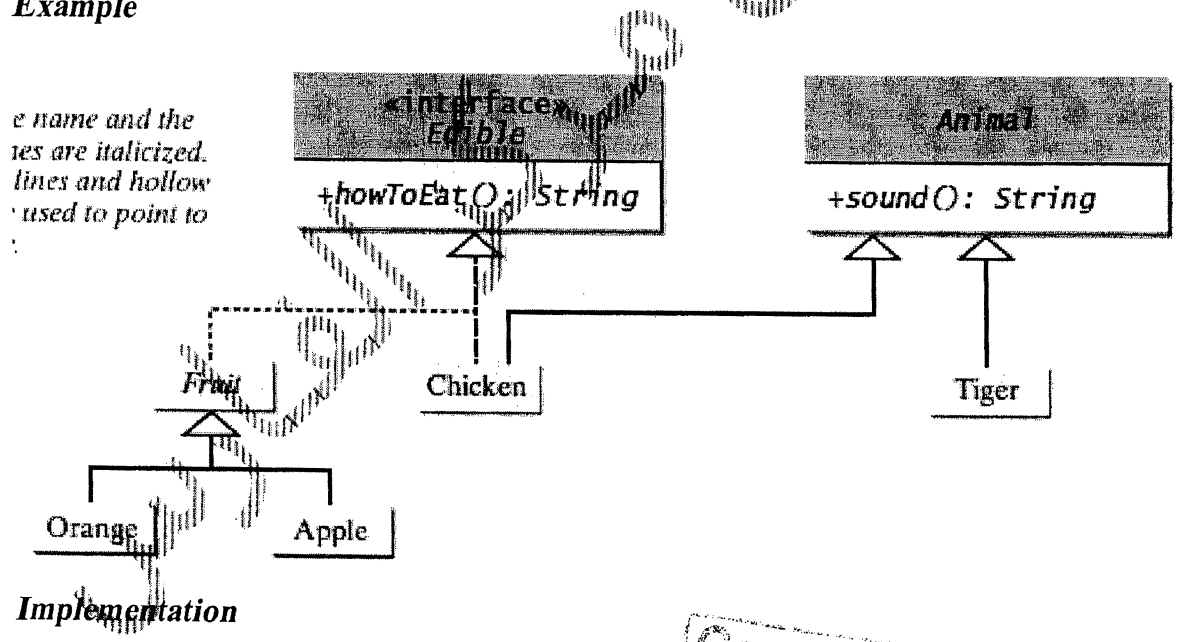
لان ال Class يمكن أن ترث Implements أكثر من Interface فى نفس الوقت ولا يمكنها الا ان ترث Extends الا فصيله واحده فقط فى نفس الوقت

أذا وجدت Class بترث من أكثر من Interface فى نفس الوقت لابد من عمل Implementation لكل ال Methods الموجوده داخل ال Interfaces

```
public class ClassName implements InterfaceName1,InterfaceName2{    }
```

يمكن لInterface أن يرث Interface أخر أو أكثر عن طريق extends

Example



Implementation

```
package testedible;
public interface Edible {
    public abstract String howToEat();
```

Center Elfares
01221733299 - 01276222288

```
}
public class TestEdible {

    public static void main(String[] args) {
        Object[] objects = {new Tiger(), new Chicken(), new Apple()};
        for (int i = 0; i < objects.length; i++) {
            if (objects[i] instanceof Edible){
                System.out.println(((Edible)objects[i]).howToEat());
            }
            if (objects[i] instanceof Animal) {
                System.out.println(((Animal)objects[i]).sound());
            }
        }
    }
}

abstract class Animal {
    public abstract String sound();
}

class Chicken extends Animal implements Edible{

    @Override
    public String sound() {
        return "Chicken: cock-a-doodle-doo";
    }

    @Override
```

Center Elfares
Center Elfares

```
    public String howToEat() {
        return "Chicken: Fry it";
    }
}

class Tiger extends Animal{

    @Override
    public String sound() {
        return "Tiger: RROOAARR";
    }
}

abstract class Fruit implements Edible {}

class Apple extends Fruit{
    @Override
    public String howToEat() {
        return "Apple: Make apple cider";
    }
}
class Orange extends Fruit {

    @Override
    public String howToEat() {
        return "Orange: Make orange juice";
    }
}
```

Center Elfares

Center Elfares

Result

```
Tiger: RROOAARR
Chicken: Fry it
Chicken: cock-a-doodle-doo
Apple: Make apple cider
```

Center Elfares
Center Elfares